UDBHAVA X’26

TEAM NAME:CIPHER

PROBLEM STATEMENT DOMAIN:FINTECH(FT-01)

PROBLEM STATEMENT NO:FT-07

INTRODUCTION:

Digital Payment Spending Control System

In recent years, digital payment systems such as UPI, credit cards, and mobile wallets have become an integral part of daily life. These technologies offer convenience, speed, and ease of transactions, encouraging users to make payments anytime and anywhere. However, this convenience has also led to a significant challenge—uncontrolled and unmonitored spending.Many users, like Arjun, rely on multiple digital platforms for their personal and business transactions. Due to the lack of real-time tracking and spending awareness, users often lose control over their expenses and exceed their budgets without realizing it. Existing banking and payment applications mainly provide transaction history but fail to offer proactive features such as real-time alerts, spending limits, and intelligent recommendations.As a result, there is a growing need for a system that not only tracks expenses but also helps users manage and control their spending behavior effectively. This problem highlights the importance of developing a Digital Payment Spending Control System that ensures better financial discipline and awareness among users.

PROBLEM STATEMENT:

- Rapid growth of digital payments (UPI, cards, wallets) has increased convenience
- Users make frequent transactions without tracking expenses
- Lack of real-time monitoring leads to overspending
- Existing apps only provide transaction history after spending
- No system to warn users before exceeding budget
- Difficulty in managing expenses across multiple platforms
- Lack of proper categorization of spending (food, travel, shopping, etc.)
- No intelligent suggestions to improve spending habits
- Users are unable to identify where most of their money is spent
- Need for a system to control and optimize spending behavior

SOLUTION:

SpendShield is not just a basic expense tracker; it is a Real-time Digital Spending Guardrail System designed to actively control and influence user spending behavior.

It acts as a "Financial Bodyguard" that continuously monitors transactions and provides intelligent feedback to prevent overspending.

Core Functionality

SpendShield performs the following key operations:

1. **Transaction Ingestion**
    - Collects transaction data in real time from multiple sources (UPI, cards, wallets using mock APIs).
2. **Smart Categorization**
    - Uses a **merchant-mapping engine** to classify expenses into categories such as food, travel, shopping, etc.
3. **Guardrail Evaluation**
    - Compares each transaction against predefined **category-wise spending limits (guardrails)**.
4. **Anomaly Detection**
    - Identifies unusual transactions based on:
        - High amount
        - Increased frequency
        - Deviation from normal spending patterns
5. **Behavioral Nudges & Alerts**
    - Provides real-time feedback using status indicators:
        - Green: Safe
        - Yellow: Warning (approaching limit)
        - Red:Breach (limit exceeded)

System Flow & Logic

◆ **Input**

- Mock API data simulating:
    - UPI transactions
    - Credit/Debit card payments
    - Wallet transactions

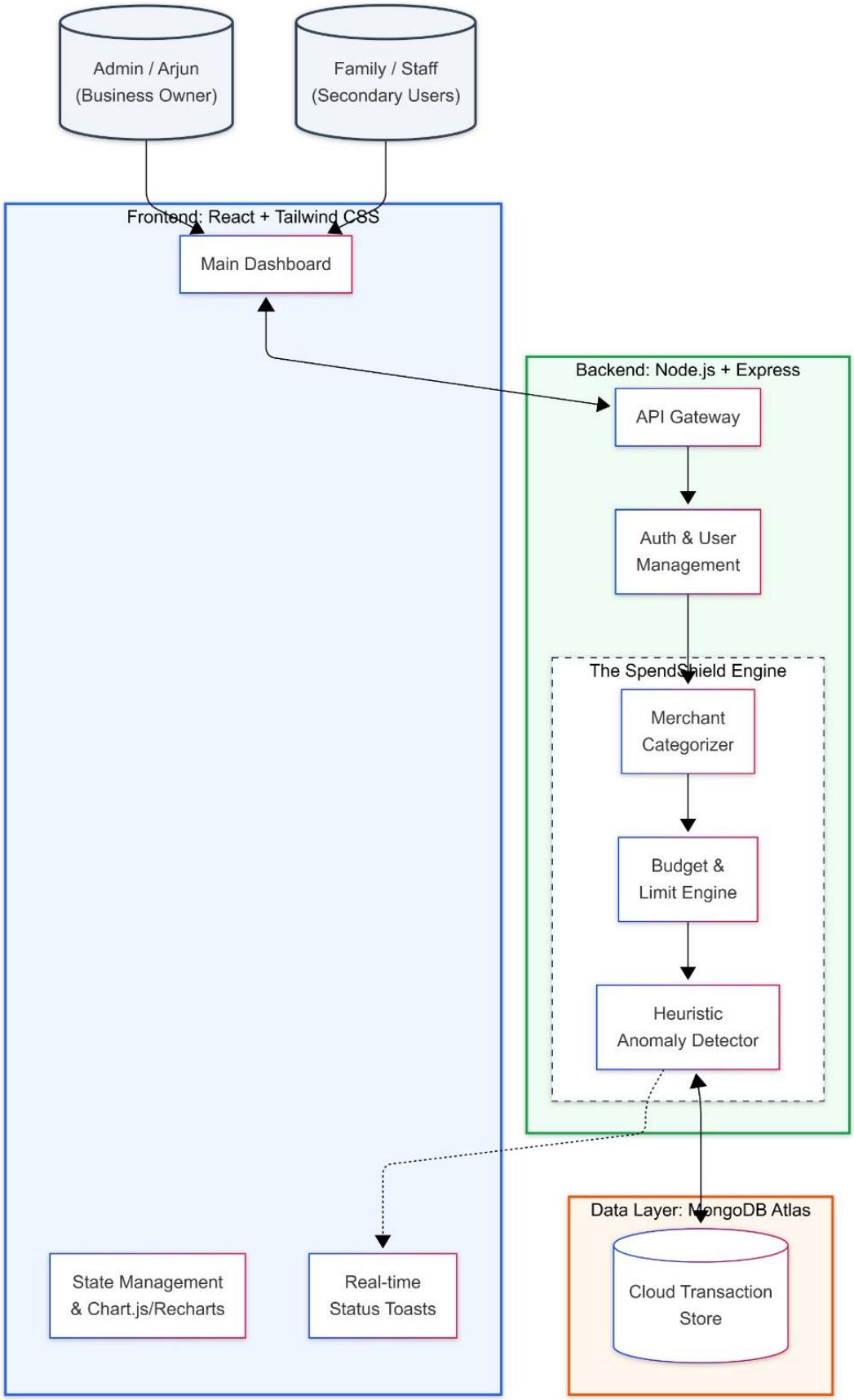
◆ **Processing**

- A **Node.js backend** performs:
    - Transaction parsing
    - Category classification
    - Budget comparison
    - Monthly spending velocity calculation
    - Alert triggering

- **Output**

  A **React-based dashboard** displays:

  - Real-time spending status
  - Category-wise usage
  - Guardrail indicators (Safe/Warning/Breach)
  - Multi-user (family/business) insights


Admin / Arjun
(Business Owner)
Family / Staff
(Secondary Users)
Frontend: React + Tailwind CSS
Main Dashboard
Backend: Node.js + Express
API Gateway
Auth & User
Management
The SpendShield Engine
Merchant
Categorizer
Budget &
Limit Engine
Heuristic
Anomaly Detector
Data Layer: MongoDB Atlas
Cloud Transaction
Store
State Management
& Chart.js/Recharts
Real-time
Status Toasts

TECHNOLOGIES USED:

1. Frontend (User Interface)

- React
  Used to build interactive dashboards and UI
- HTML, CSS, JavaScript
  For designing layout and styling

2. Backend (Server Logic)

- Node.js
  Handles server-side logic and API requests
- Express.js
  Builds REST APIs and manages routes

3. Database

- MongoDB Atlas
  Stores:
  - User data
  - Transactions
  - Budgets
  - Alerts

4. Data Processing & Logic

- JavaScript functions
  For:
  - Categorization
  - Budget checking
  - Alert generation

5. Visualization

- Chart.js
  Displays graphs like:
  - Monthly spending

- Category-wise analysis

LIMITATIONS

1.Basic Categorization

- Merchant mapping is **rule-based**
- May misclassify unknown or new merchants

2.No Automatic Payment Control

- Can **alert users**, but cannot block transactions
- User must take action manually